UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Metodología de Sistemas II

Trabajo Práctico Integrador

Evolución profesional de un sistema existente: calidad, deuda técnica, refactoring y revisión técnica

Sistema de Gestión de Solicitudes Académicas

Integrantes: Gevont Utmazian, Martín Maine
Comisión: M25 C4-08
Año académico: 2026

Fecha de entrega: 08/11/2026.

Informe técnico. El uso de IA se detalla en el apartado correspondiente.

Índice

1. Resumen ejecutivo 3

2. Introducción y objetivos 3

3. Sistema base y alcance 4

4. Calidad externa e interna 5

5. Deuda técnica y code smells 6

6. SOLID, Clean Code y decisiones 7

7. Refactoring seguro 8

8. Pruebas y verificación 12

9. Code review y criterios de aceptación 16

10. Conclusiones y mejoras futuras 18

11. Uso de inteligencia artificial 19

12. Referencias y recursos 20

13. Auditoría y cierre de entrega 21

Anexo A. Historial real de desarrollo 22

Anexo B. Prompt maestro suministrado 23

1. Resumen ejecutivo

Se construyó un prototipo local para gestionar solicitudes académicas.
La versión inicial resolvía los casos previstos, pero concentraba reglas y SQL.
El diagnóstico identificó ocho debilidades de legibilidad, cohesión y acoplamiento.
Se aplicaron cuatro refactorizaciones y se conservó la interfaz de consola.
Las 29 pruebas compartidas aprobaron por hito; la revisión final alcanzó 33.
Diez pasos de consola dieron resultados idénticos entre las dos versiones.

2. Introducción y objetivos

El trabajo se centra en mejorar un módulo acotado sin ampliar sus funciones por acumulación. Para ello se creó un prototipo expresamente para esta actividad. No se lo presenta como un sistema institucional anterior ni como una solución de producción. Su organización inicial, monolítica y simple, permite analizar dificultades reales de mantenimiento sin introducir fallos funcionales a propósito.

El objetivo general es mostrar una relación verificable entre diagnóstico, cambio y prueba. Los objetivos concretos son separar responsabilidades, reducir duplicaciones, simplificar datos de entrada y explicitar las reglas de estados. Cada mejora se vincula con una debilidad y con registros de ejecución reales.

Este documento corresponde al trabajo grupal de Gevont Utmazian y Martín Maine, comisión M25 C4-08. La generación contó con asistencia sustancial de IA. La revisión personal de cada integrante, los enlaces externos, el video y la defensa todavía requieren intervención de los estudiantes. No se atribuyen autorizaciones ni aprendizajes personales no comprobados.

2.1 Integrantes, modalidad y participación

La modalidad es grupal. La tabla recoge el reparto de participación y las tareas de cada integrante informados por el equipo.

Integrante	Participación declarada	Detalle de aportes
Gevont Utmazian	56 %	Diseño del sistema y de la persistencia en SQLite. Diagnóstico de los problemas de diseño, aplicación de las refactorizaciones y comparación del comportamiento entre versiones.
Martín Maine	44 %	Descripción del caso de uso y del alcance. Preparación de ejemplos de consola, organización de las evidencias de prueba y revisión de la documentación de uso.

3. Sistema base y alcance

El sistema representa el registro de solicitudes por parte de un operador de una institución educativa. El estudiante es el solicitante; el responsable representa a una persona o área que atiende el trámite. En este prototipo ninguno inicia sesión: el operador usa una consola local. No se confundieron actores del dominio con cuentas autenticadas.

Estructura	Datos principales	Relación
Estudiante	id, nombre, email único	Uno a muchas solicitudes
Responsable	id, nombre	Cero o uno por solicitud
Solicitud	estudiante, tipo, prioridad, asunto, descripción, estado, responsable	Referencias protegidas por claves foráneas
Estado	pendiente, en_proceso, resuelta, cancelada	Valor del dominio; no tabla adicional

Se implementan ocho operaciones de consola: registrar estudiante, registrar responsable, registrar solicitud, asignar, cambiar estado, consultar, listar con filtros y reportar. Tipos: certificado, equivalencia e inscripcion. Prioridades: baja, media y alta. Todas las solicitudes nuevas comienzan pendientes.

No se incluyen adjuntos, autenticación, eliminación, notificaciones, plazos ni uso simultáneo multiusuario. La prioridad clasifica, pero no ordena automáticamente una cola. El reporte cuenta solicitudes por estado. La búsqueda opera sobre el asunto y trata el texto de forma literal.

3.1 Tecnología y ejecución

Python permite resolver consola, pruebas y persistencia con su biblioteca estándar. SQLite usa un archivo local y no requiere servidor [R2]. unittest organiza fixtures y casos automatizados [R1]. Se usó Python 3.10.0 disponible en el equipo; no se instaló pytest porque el intento de descarga fue bloqueado por la red. Esta adaptación fue técnica, no un requisito de la cátedra.

```
python -m unittest discover -s tests -v
python demo.py
python cli.py --db prueba.db reporte
```

La versión inicial se identifica con estado-inicial y se conserva también en versiones/inicial. El esquema de base no cambia, por lo que la comparación no necesita migraciones. Los datos usados en la demostración son ficticios.

4. Calidad externa e interna

4.1 Comparación de calidad externa e interna

Situación	Observación externa	Observación interna
Registrar una solicitud válida	Devuelve un identificador y conserva los datos	Una rama combina consultas previas, validaciones y SQL de inserción
Rechazar datos incompletos	Mensaje específico y ninguna solicitud persistida	La validación de texto se repite
Cambiar estado	Impide saltar de pendiente a resuelta	Las reglas están embebidas en una expresión booleana
Filtrar y reportar	Devuelve filas ordenadas y totales consistentes	Literales y acceso a datos viven en el mismo despachador

Las pruebas externas caracterizan el comportamiento, no certifican por sí solas mantenibilidad, rendimiento o seguridad integral. Los 29 casos iniciales se ejecutaron con `python -m unittest discover -s tests -v`; el registro auténtico está en `evidencias/pruebas/01_inicial.txt`.

4.2 Relación con la evolución futura

Desde el punto de vista del usuario, la versión inicial es correcta en los casos cubiertos: registra, asigna, cambia estado, filtra y reporta. La dificultad aparece al cambiar el sistema. Modificar el ciclo de estados exige editar una expresión booleana dentro de `operar`; agregar una operación obliga a ampliar el mismo despachador; y probar una regla requiere montar SQLite. Esos costos no se ven en la consola, pero determinan cuánto cuesta cada cambio y cuánto riesgo conlleva. La sección 5 detalla las ocho debilidades que explican esa diferencia y la sección 7 muestra cómo se redujeron.

5. Deuda técnica y code smells

5.1 Diagnóstico documentado

La referencia estable es `estado-inicial:sistema.py`. El prototipo fue construido para el TPI con una organización monolítica deliberadamente simple. No se presenta como software heredado de una institución. Funciona en los 29 casos de contrato ejecutados, pero modificarlo exige conocer SQL, reglas de negocio y el despachador de operaciones en una misma clase.

ID	Debilidad y ubicación	Evidencia literal	Impacto y riesgo	Prioridad	Corrección prevista
CS01	Long Method: `Sistema.operar`	`def operar(self, op, **d)` contiene todas las ramas	Una edición puede afectar operaciones no relacionadas; cuesta revisar el flujo completo	Alta	R1 Extract Method
CS02	Duplicación: ramas estudiante/responsable	`if not isinstance(d.get("nombre"), str) or not d["nombre"].strip()`	Dos validaciones iguales pueden divergir	Media	R1 función de texto obligatorio
CS03	Clase con responsabilidades mezcladas: `Sistema`	`sqlite3.connect(ruta)` y `raise ValueError("Transición inválida")`	Persistencia y política de estados cambian por razones diferentes	Alta	R2 Move Method, R4 Extract Class
CS04	Long Parameter List: `registrar_solicitud`	`estudiante_id, tipo, prioridad, asunto, descripcion, responsable_id=None`	Es fácil intercambiar cadenas posicionales del mismo tipo	Media	R3 Introduce Parameter Object
CS05	Literales de dominio repetidos: solicitud/listar	`("baja", "media", "alta")`	Un valor nuevo requiere cambios en puntos separados	Media	R4 constantes compartidas
CS06	Condicional compuesto: cambio de estado	`(r["estado"] == "pendiente" and ...) or (...)`	Al ampliar el ciclo es fácil habilitar una transición incorrecta	Alta	R4 tabla de transiciones
CS07	Nombres abreviados: listado y conexión	`q`, `p`, `r`, `d`, `self.c`	La lectura depende de reconstruir el significado local	Baja	R1 nombres de parámetros y R2 repositorio
CS08	Acoplamiento a la base de datos: creación y consultas	`self.c.execute(...)` dentro de reglas de negocio	Probar una política exige montar SQLite; cambiar persistencia afecta el servicio	Alta	R2 acceso encapsulado y R4 inyección de repositorio

CS03 describe la falta de cohesión de la clase; CS08, la dependencia concreta de sus reglas. Están relacionados, pero sus consecuencias y verificaciones son diferentes. Ninguno implica que el programa sea inseguro por definición: las consultas ya usan parámetros y las claves foráneas están activadas.

La prioridad alta se reserva para cambios que afectan reglas o varias operaciones. La media corresponde a duplicaciones y firmas que multiplican el costo del cambio. La baja corresponde a legibilidad localizada. No se asignaron tiempos ni costos económicos ficticios.

5.2 Ubicaciones comprobables

CS01 - versiones/inicial/sistema.py:31

```
def operar(self, op, **d):
    if op == "estudiante":
        if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
```

CS02 - versiones/inicial/sistema.py:33

```
            if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
                raise ValueError("Nombre obligatorio")
            if not isinstance(d.get("email"), str) or "@" not in d["email"] or not d["email"].strip():
```

CS03 - versiones/inicial/sistema.py:7

```
        self.c = sqlite3.connect(ruta)
        self.c.row_factory = sqlite3.Row
        self.c.execute("PRAGMA foreign_keys = ON")
```

CS04 - versiones/inicial/sistema.py:25

```
    def registrar_solicitud(self, estudiante_id, tipo, prioridad, asunto, descripcion,
                            responsable_id=None):
        return self.operar("solicitud", estudiante_id=estudiante_id, tipo=tipo,
```

CS05 - versiones/inicial/sistema.py:55

```
            if d.get("prioridad") not in ("baja", "media", "alta"):
                raise ValueError("Prioridad inválida")
            if not isinstance(d.get("asunto"), str) or not d["asunto"].strip():
```

CS06 - versiones/inicial/sistema.py:83

```
            if not ((r["estado"] == "pendiente" and d.get("estado") in ("en_proceso", "cancelada"))
                    or (r["estado"] == "en_proceso" and d.get("estado") in ("resuelta", "cancelada"))):
                raise ValueError("Transición inválida")
```

CS07 - versiones/inicial/sistema.py:95

```
            q = "SELECT * FROM solicitudes WHERE 1=1"
            p = []
            if d.get("estado") is not None:
```

CS08 - versiones/inicial/sistema.py:51

```
            if not self.c.execute("SELECT id FROM estudiantes WHERE id=?",
    (d.get("estudiante_id"),)).fetchone():
                raise ValueError("Estudiante inexistente")
            if d.get("tipo") not in ("certificado", "equivalencia", "inscripcion"):
```

6. SOLID, Clean Code y decisiones

SOLID aplicado con límites explícitos

Principio	Problema y cambio	Alcance real
SRP	Sistema mezclaba SQL y reglas. RepositorioSQLite administra almacenamiento; dominio concentra reglas; cli presenta resultados	Aplicado directamente. El servicio coordina casos de uso
OCP	El ciclo de estados estaba escrito como expresión booleana. TRANSICIONES permite cambiar relaciones sin editar el algoritmo de validación	Mejora localizada: una operación nueva todavía exige modificar el despachador. No se afirma cumplimiento absoluto
LSP	No hay jerarquía de herencia de producción que requiera sustitución de subclases	No se fuerza. La prueba de un repositorio mínimo demuestra sustitución solamente para reporte, no equivalencia total de repositorios
ISP	No se agregó una interfaz universal con operaciones ajenas al caso de uso	No se introducen interfaces nominales. El reporte utiliza únicamente contar_por_estado; una división mayor sería innecesaria para este alcance
DIP	El constructor acepta un repositorio; las reglas ya no ejecutan SQL	Aplicación parcial mediante inyección y contrato estructural. Sistema todavía importa SQLite como opción predeterminada; un composition root independiente sería una mejora futura

Clean Code se aplica mediante nombres descriptivos, validación reutilizable, métodos acotados y mensajes estables. Se mantiene una firma antigua con seis parámetros como adaptador: retirarla rompería compatibilidad. El código nuevo puede usar guardar_solicitud(NuevaSolicitud(...)) con nombres explícitos.

Decisiones técnicas

1. Python y SQLite permiten ejecutar el sistema sin servidor ni servicios externos. Las tres tablas y sus relaciones cubren el alcance sin un ORM.

2. unittest reemplazó la propuesta inicial pytest: pytest no estaba instalado y la instalación falló por restricciones de red. Se conservaron los mismos casos e intenciones. No se declara una ejecución de pytest.

3. Python 3.10.0 fue el intérprete disponible y efectivamente usado. La propuesta 3.11 no era un requisito obligatorio. No se recomienda instalar una versión antigua para producción.

4. Las transacciones cubren cada escritura. No se implementó un flujo transaccional multiusuario entre lectura y actualización: la aplicación es local, de un operador.

5. Se mantienen validación básica de email, búsqueda SQL y reglas iniciales. Corregirlas ahora sería un cambio funcional que debe separarse de la refactorización.

6. No se eliminó la API operar ni se alteró el esquema de SQLite. Las salidas JSON de diez pasos fueron idénticas entre ambas consolas.

7. Los datos de ejemplo son sintéticos. El nombre académico sólo aparece en documentación y no se incorpora como dato de estudiante del sistema.

8. No se calcularon cobertura ni métricas de complejidad ciclomática con herramientas externas. Las métricas estructurales del anexo se obtienen con ast y no son una puntuación de calidad.

Relaciones y componentes

Un estudiante puede tener varias solicitudes. Una solicitud tiene exactamente un estudiante y cero o un responsable. Un responsable puede atender varias solicitudes. La prioridad y el tipo clasifican; el estado representa el avance. La consola llama al servicio; éste utiliza reglas de dominio y el repositorio; únicamente el repositorio conoce SQL.

7. Refactoring seguro

Las cuatro transformaciones se hicieron en commits reales, en el orden del historial. Las pruebas preceden a los cambios y se repiten por hito. Los fragmentos siguientes se extraen del código; cuando son parciales se indica su rango. No se alteraron los registros históricos para aparentar trabajo previo.

R1 - Extract Method y validación compartida

CS01, CS02 y parte de CS07. Commit 32d2492. Se dividió el despachador en operaciones y se extrajo la validación repetida del nombre. SRP y legibilidad: cada caso de uso se inspecciona por separado. La interfaz operar y los errores permanecieron. Ver 02_extract_method.txt: 29 pruebas aprobadas.

versiones/inicial/sistema.py, líneas 31-45.

```
    def operar(self, op, **d):
        if op == "estudiante":
            if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
                raise ValueError("Nombre obligatorio")
            if not isinstance(d.get("email"), str) or "@" not in d["email"] or not d["email"].strip():
                raise ValueError("Email inválido")
            try:
                with self.c:
                    r = self.c.execute("INSERT INTO estudiantes(nombre,email) VALUES (?,?)",
                                       (d["nombre"].strip(), d["email"].strip().lower()))
                return r.lastrowid
            except sqlite3.IntegrityError as exc:
                raise ValueError("Email ya registrado") from exc
        elif op == "responsable":
            if not isinstance(d.get("nombre"), str) or not d["nombre"].strip():
```

sistema.py, líneas 19-28.

```
    def operar(self, operacion, **datos):
        acciones = {
            "estudiante": self._estudiante, "responsable": self._responsable,
            "solicitud": self._solicitud, "asignar": self._asignar,
            "estado": self._estado, "consultar": self._consultar,
            "listar": self._listar, "reporte": self._reporte,
        }
        if operacion not in acciones:
            raise ValueError("Operación desconocida")
        return acciones[operacion](datos)
```

dominio.py, líneas 25-27.

```
def validar_texto(valor, mensaje):
    if not isinstance(valor, str) or not valor.strip():
        raise ValueError(mensaje)
```

R2 - Move Method / reubicación de persistencia

CS03 y CS08. Commit 1cfd75c: conexión, esquema y transacciones se movieron al repositorio. El registro 03_move_method.txt conserva 29 aprobadas. La revisión detectó que el servicio todavía contenía SQL; a379c39 completó el encapsulamiento con métodos semánticos. Esa diferencia se declara para no atribuir al primer paso una separación que todavía no tenía. SRP y DIP parcial.

versiones/inicial/sistema.py, líneas 6-20.

```
    def __init__(self, ruta=":memory:"):
        self.c = sqlite3.connect(ruta)
        self.c.row_factory = sqlite3.Row
        self.c.execute("PRAGMA foreign_keys = ON")
        self.c.executescript("""
        CREATE TABLE IF NOT EXISTS estudiantes (
            id INTEGER PRIMARY KEY, nombre TEXT NOT NULL, email TEXT NOT NULL UNIQUE);
        CREATE TABLE IF NOT EXISTS responsables (
            id INTEGER PRIMARY KEY, nombre TEXT NOT NULL);
        CREATE TABLE IF NOT EXISTS solicitudes (
            id INTEGER PRIMARY KEY, estudiante_id INTEGER NOT NULL REFERENCES estudiantes(id),
            tipo TEXT NOT NULL, prioridad TEXT NOT NULL, asunto TEXT NOT NULL,
            descripcion TEXT NOT NULL, estado TEXT NOT NULL,
            responsable_id INTEGER REFERENCES responsables(id));
        """)
```

repositorio.py, líneas 42-47.

```
    def agregar_solicitud(self, nueva):
        return self._insertar("""INSERT INTO solicitudes
            (estudiante_id,tipo,prioridad,asunto,descripcion,estado,responsable_id)
            VALUES (?,?,?,?,?,'pendiente',?)""",
            (nueva.estudiante_id, nueva.tipo, nueva.prioridad,
             nueva.asunto.strip(), nueva.descripcion.strip(), nueva.responsable_id))
```

sistema.py, líneas 7-8.

```
    def __init__(self, ruta=":memory:", repositorio=None):
        self.repositorio = repositorio if repositorio is not None else RepositorioSQLite(ruta)
```

R3 - Introduce Parameter Object

CS04. Commit b5eb55f. NuevaSolicitud reúne los seis datos de una solicitud; guardar_solicitud recibe ese objeto. La API posicional original se conserva como adaptador, por lo que la simplificación no exige romper clientes. No se presenta como eliminación total de la firma larga. Registro 04_parameter_object.txt: 29 aprobadas. La prueba adicional test_objeto_parametros comprueba la nueva vía.

versiones/inicial/sistema.py, líneas 25-29.

```
    def registrar_solicitud(self, estudiante_id, tipo, prioridad, asunto, descripcion,
                            responsable_id=None):
        return self.operar("solicitud", estudiante_id=estudiante_id, tipo=tipo,
                           prioridad=prioridad, asunto=asunto, descripcion=descripcion,
                           responsable_id=responsable_id)
```

dominio.py, líneas 16-22.

```
class NuevaSolicitud:
    estudiante_id: int
    tipo: str
    prioridad: str
    asunto: str
    descripcion: str
    responsable_id: int | None = None
```

sistema.py, líneas 46-58.

```
    def guardar_solicitud(self, nueva: NuevaSolicitud):
        if not self.repositorio.existe_estudiante(nueva.estudiante_id):
            raise ValueError("Estudiante inexistente")
        if nueva.tipo not in TIPOS:
            raise ValueError("Tipo inválido")
        if nueva.prioridad not in PRIORIDADES:
            raise ValueError("Prioridad inválida")
        validar_texto(nueva.asunto, "Asunto obligatorio")
        validar_texto(nueva.descripcion, "Descripción obligatoria")
        if nueva.responsable_id is not None:
            if not self.repositorio.existe_responsable(nueva.responsable_id):
                raise ValueError("Responsable inexistente")
        return self.repositorio.agregar_solicitud(nueva)
```

R4 - Decompose Conditional y reglas separadas

CS05 y CS06; completa CS03, CS07 y CS08. Commit a379c39. La condición compuesta se reemplazó por una consulta a TRANSICIONES, con catálogos compartidos. La creación del repositorio también materializa Extract Class. Se preservan exactamente las cuatro transiciones permitidas. OCP localizado: cambia la tabla, no el algoritmo. Registro 05_separar_reglas.txt: 29 aprobadas; la revisión posterior recorre los 16 pares.

versiones/inicial/sistema.py, líneas 79-89.

```
        elif op == "estado":
            r = self.c.execute("SELECT * FROM solicitudes WHERE id=?", (d.get("id"),)).fetchone()
            if r is None:
                raise ValueError("Solicitud inexistente")
            if not ((r["estado"] == "pendiente" and d.get("estado") in ("en_proceso", "cancelada"))
                    or (r["estado"] == "en_proceso" and d.get("estado") in ("resuelta", "cancelada"))):
                raise ValueError("Transición inválida")
            with self.c:
                self.c.execute("UPDATE solicitudes SET estado=? WHERE id=?", (d["estado"], d["id"]))
            return None
        elif op == "consultar":
```

dominio.py, tabla de transiciones.

```
TRANSICIONES = {
    "pendiente": ("en_proceso", "cancelada"),
    "en_proceso": ("resuelta", "cancelada"),
    "resuelta": (),
    "cancelada": (),
}
ESTADOS = tuple(TRANSICIONES)
```

dominio.py, líneas 30-32.

```
def validar_transicion(actual, destino):
    if destino not in TRANSICIONES.get(actual, ()):
        raise ValueError("Transición inválida")
```

7.5 Indicadores estructurales

Versión	Líneas de operar	Funciones en sistema.py
Inicial	88	4
Final	10	13

Se contaron líneas físicas del nodo AST y funciones declaradas. El número total de funciones puede aumentar al extraer responsabilidades. Estos datos no prueban una mejora de rendimiento ni constituyen una métrica de calidad por sí solos.

8. Pruebas y verificación

Los identificadores P01-P29 organizan los casos por funcionalidad; unittest los imprime ordenados por nombre de método. Antes: registro 01_inicial.txt. Después: 05_separar_reglas.txt y 07_revision_optimizada.txt. En las columnas se indica coincidencia con el resultado esperado, no salidas fabricadas.

ID	Caso y entrada	Resultado esperado	Antes	Después
P01	Estudiante válido: Demo; demo@example.test	ID 1	Coincide	Coincide
P02	Email duplicado: Mismo email con mayúsculas	Email ya registrado	Coincide	Coincide
P03	Nombre vacío: nombre vacío	Nombre obligatorio	Coincide	Coincide
P04	Email inválido: incorrecto	Email inválido	Coincide	Coincide
P05	Solicitud válida: Certificado, alta, estudiante 1	Fila completa; pendiente	Coincide	Coincide
P06	Asunto vacío: espacios	Asunto obligatorio; no persiste	Coincide	Coincide
P07	Descripción vacía: cadena vacía	Descripción obligatoria; no persiste	Coincide	Coincide
P08	Tipo inválido: otro	Tipo inválido; no persiste	Coincide	Coincide
P09	Prioridad inválida: urgente	Prioridad inválida; no persiste	Coincide	Coincide
P10	Estudiante inexistente: ID 999	Estudiante inexistente	Coincide	Coincide
P11	Responsable inicial inexistente: ID 999	Responsable inexistente	Coincide	Coincide
P12	Asignar responsable: Solicitud 1; responsable 1	responsable_id=1	Coincide	Coincide
P13	Responsable al crear: Responsable válido	Asignación persistida	Coincide	Coincide
P14	Ciclo normal: pendiente; en_proceso; resuelta	Dos cambios válidos	Coincide	Coincide
P15	Salto inválido: pendiente a resuelta	Transición inválida; conserva estado	Coincide	Coincide
P16	Estado repetido: pendiente a pendiente	Transición inválida	Coincide	Coincide
P17	Estado desconocido: desconocido	Transición inválida	Coincide	Coincide
P18	Cancelación terminal: cancelada a en_proceso	Transición inválida	Coincide	Coincide
P19	Filtros y búsqueda: alta, pendiente, CONSTANCIA y %	Resultados combinados y búsqueda literal	Coincide	Coincide
P20	Filtro estado inválido: otro	Estado inválido	Coincide	Coincide
P21	Filtro prioridad inválida: otra	Prioridad inválida	Coincide	Coincide
P22	Reporte: Base vacía y una pendiente	Total 0 y 1; estados en cero incluidos	Coincide	Coincide
P23	Consulta inexistente: ID 999	Solicitud inexistente	Coincide	Coincide
P24	Cambio inexistente: ID 999	Solicitud inexistente	Coincide	Coincide
P25	Asignación inexistente: ID 999	Solicitud inexistente	Coincide	Coincide
P26	Responsable al asignar inexistente: ID 999	Rechazo; mantiene sin responsable	Coincide	Coincide
P27	Persistencia: Cerrar y reabrir archivo	Mismo asunto y solicitud	Coincide	Coincide

ID	Caso y entrada	Resultado esperado	Antes	Después
P28	SQL como dato: Texto con DROP TABLE	Se guarda literal; tabla conservada	Coincide	Coincide
P29	Operación desconocida: borrar_todo	Operación desconocida	Coincide	Coincide

Verificaciones añadidas en revisión

R01 recorre 16 pares de estados, con cuatro permitidos y doce rechazados. R02 prueba NuevaSolicitud. R03 inyecta un doble de conteos para reporte sin SQLite. R04 ejecuta una CLI con ID inexistente y verifica JSON, código 1 y ausencia de stderr. Estas cuatro pruebas no existían en la suite inicial; no se presentan como comparación antes/después.

La versión final ejecutó 33 pruebas. La matriz de 16 pares cuenta como un método con subcasos. demo.py realiza además diez pasos por versión y compara salida y código. No se midió cobertura; no se promete verificación exhaustiva.

Método

Cada prueba de servicio abre SQLite en memoria y cierra la conexión. La persistencia se prueba con un archivo temporal reabierto. Las pruebas de consola usan procesos reales y bases temporales. No se usa una base de producción ni se limpian archivos del usuario. Los registros son salidas capturadas de los comandos ejecutados; no son capturas recreadas gráficamente.

8.1 Registros de ejecución

01_inicial.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.128s

OK
```

02_extract_method.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.102s

OK
```

03_move_method.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.106s

OK
```

04_parameter_object.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.112s

OK
```

05_separar_reglas.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 29 tests in 0.104s

OK
```

06_revision_final.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 33 tests in 0.393s

OK
```

07_revision_optimizada.txt

Extracto final del registro auténtico; archivo completo incluido en el anexo ZIP.

```
----------------------------------------------------------------------
Ran 33 tests in 0.382s

OK
```

8.2 Demostración antes/después

La ejecución de demo.py informó IGUALES para diez pasos. Ejemplos de resultados realmente capturados:


```
{
  "argumentos": [
    "solicitud",
    "--estudiante-id",
    "1",
    "--tipo",
    "certificado",
    "--prioridad",
    "alta",
    "--asunto",
    "Constancia regular",
    "--descripcion",
    "Beca ficticia"
  ],
  "salida": {
    "ok": true,
    "resultado": 1
  },
  "codigo": 0
}

{
  "argumentos": [
    "consultar",
    "--id",
    "1"
  ],
  "salida": {
    "ok": true,
    "resultado": {
      "id": 1,
      "estudiante_id": 1,
      "tipo": "certificado",
      "prioridad": "alta",
      "asunto": "Constancia regular",
      "descripcion": "Beca ficticia",
      "estado": "resuelta",
      "responsable_id": 1
    }
  },
  "codigo": 0
}
```

```
{
  "argumentos": [
    "estado",
    "--id",
    "1",
    "--estado",
    "pendiente"
  ],
  "salida": {
    "ok": false,
    "error": "Transici├│n inv├ílida"
  },
  "codigo": 1
}

{
  "argumentos": [
    "reporte"
  ],
  "salida": {
    "ok": true,
    "resultado": {
      "total": 1,
      "por_estado": {
        "pendiente": 0,
        "en_proceso": 0,
        "resuelta": 1,
        "cancelada": 0
      }
    }
  },
  "codigo": 0
}
```

9. Code review y criterios de aceptación

Autorrevisión técnica asistida por IA

Revisión del artefacto por el mismo asistente que lo generó, el 01/11/2026. No constituye revisión independiente ni aprobación de un compañero o docente. Con dos integrantes corresponde registrar antes de entregar una revisión cruzada real (cada uno revisa el aporte del otro en el repositorio) en docs/checklist_code_review.md; hasta entonces consta sólo la autorrevisión asistida.

Criterio	Resultado	Evidencia o límite
Legibilidad y nombres	Conforme con límite	Métodos descriptivos; API compatible conserva diccionarios
Responsabilidades y cohesión	Conforme	Consola, servicio, dominio y repositorio separados
Duplicación	Conforme con límite	Validación de texto común; catálogo centralizado. Literal pendiente sigue en SQL de inserción
Complejidad	Conforme al alcance	Operaciones separadas; estados definidos por tabla
Validaciones	Conforme al contrato	Rechazos no crean solicitudes; mensajes conservados
Manejo de errores	Parcial	Errores de dominio controlados; fallos de disco/SQLite no traducidos en la CLI
Acoplamiento y SOLID	Parcial justificado	Inyección disponible; dependencia predeterminada SQLite permanece
Persistencia	Conforme	Reapertura probada, claves foráneas y transacciones
Seguridad básica	Conforme al alcance	SQL parametrizado, datos sintéticos; sin autenticación
Pruebas	Conforme	29 casos de contrato y 4 revisiones; matriz de 16 pares de estados
Organización	Conforme	Versión inicial conservada; evidencias separadas
Documentación	Conforme al contenido local	README, diagnóstico, refactorings, IA, defensa e informe
Mantenibilidad	Mejorada con deuda residual	No se atribuye escalabilidad ni preparación para producción
Entrega externa	Pendiente	GitHub, video y confirmación administrativa

Observaciones y acciones

ID	Archivo	Severidad	Observación y recomendación	Acción y estado
CR01	sistema.py, hito Move Method	Media	Mover sólo conexión dejaba SQL en las reglas. Encapsular también consultas	Corregido en a379c39: métodos semánticos del repositorio
CR02	tests/test_contrato.py	Media	Suite de contrato no recorre todos los pares de estados	Se añadió test_matriz_completa_transiciones: 16 subcasos, sin cambiar producción
CR03	cli.py	Media	Las pruebas del servicio no garantizan salida de consola	demo.py compara diez pasos reales; prueba adicional verifica error y código 1
CR04	sistema.py	Baja	Inyección sin prueba de sustitución	Se agregó doble mínimo para reporte; no se lo presenta como repositorio completo
CR05	dominio.py, repositorio.py	Baja	Estado inicial pendiente también aparece como literal SQL	Deuda residual aceptada; no se inventa eliminación total de literales
CR06	cli.py	Media	Error de disco/base dañada puede generar traceback	Pendiente de mejora funcional independiente; documentado en README
CR07	tests/test_contrato.py	Media	Asserts nativos pueden omitirse con python -O	Reemplazados por aserciones unittest y ejecutada la suite con optimización

Criterios de aceptación técnica

La suite compartida debe aprobar en el estado inicial y en cada refactoring. La demostración debe devolver igualdad exacta. No debe existir SQL en el servicio final. El ZIP debe poder extraerse y ejecutar pruebas sin instalar paquetes. Deben existir los seis tags y un historial recuperable. Los pendientes externos deben permanecer visibles en el informe.

El resultado local no se presenta como una entrega académica completa mientras falten enlaces y video personal.

10. Conclusiones y mejoras futuras

El resultado más relevante es la separación entre reglas de aplicación y acceso a datos. Antes, el mismo método contenía validaciones, consultas y cambios de estado. Después, el servicio expresa las operaciones y el repositorio administra las consultas. La tabla de transiciones permite inspeccionar el ciclo sin reconstruir una expresión booleana extensa.

El proceso también mostró que una extracción superficial puede no resolver el problema completo. Mover únicamente la conexión dejó SQL dentro del servicio; fue necesario encapsular las consultas para completar esa mejora. La revisión de pruebas detectó otra debilidad: las aserciones nativas podían omitirse con optimización. Se sustituyeron y se verificó la suite con python -O.

No se afirma que el sistema esté libre de errores. La evidencia respalda el contrato cubierto. Quedan pendientes mensajes para errores técnicos, validación de email más estricta, búsqueda Unicode completa, autenticación y concurrencia. Esos cambios deberían planificarse como mejoras funcionales separadas y con pruebas propias.

Conclusión grupal

Borrador para acordar entre los integrantes: «El trabajo mostró que un sistema puede cumplir su contrato externo y aun así concentrar demasiados motivos de cambio. Separar servicio, dominio y repositorio, y explicitar las transiciones de estado, hizo más localizable cada modificación; las pruebas compartidas entre ambas versiones permitieron controlar que el comportamiento observable se mantuviera. Los límites son los ya señalados: autorrevisión asistida, errores técnicos sin traducir y validaciones básicas.»

Conclusiones individuales

Gevont Utmazian. Borrador para revisar tras estudiar y ejecutar el proyecto: «La comparación muestra que un programa puede dar resultados correctos y aun así ser difícil de mantener. Separar las reglas y el almacenamiento ayuda a ubicar los cambios, y las pruebas permiten comprobar que esa reorganización conserva el comportamiento esperado. El criterio principal no fue sumar funciones, sino poder justificar las mejoras con ejemplos y resultados.»

Martín Maine. Borrador para revisar tras estudiar y ejecutar el proyecto: «Lo central fue distinguir entre que el programa funcione y que pueda modificarse sin riesgo. El diagnóstico con ubicación, impacto y prioridad permitió decidir qué corregir primero, y la comparación de diez pasos de consola entre las dos versiones dio una evidencia concreta de que el refactoring no cambió lo que ve el usuario.»

Estos párrafos son borradores: cada integrante debe reemplazarlos o ampliarlos con su reflexión real. No certifican un aprendizaje ya demostrado.

11. Uso de inteligencia artificial

Herramienta: ChatGPT/Codex en Work Mode. Fecha conocida: 01/11/2026. Intervención sustancial: lectura de requisitos, diseño del prototipo, programación, refactorización, generación y ejecución de pruebas, revisión técnica, documentación y preparación de defensa. No se limita a corrección ortográfica.

Intercambios disponibles

1. Prompt maestro suministrado por el usuario: copia íntegra del texto extraído en `prompt_maestro.txt` (se normalizaron saltos de línea). Fuente: archivo Texto pegado(7).txt. Contiene alcance, criterios de redacción, etapas, evidencias y prohibición de publicar sin autorización.

2. Instrucción del usuario en esta sesión: «Ahi te cargue el tpi, te di el prompt maestro para que empieces a trabajar en ello.» Se utilizó como autorización para construir y verificar los entregables locales.

3. Respuesta de ejecución del asistente: lectura completa de consigna y complementos; selección del caso sugerido. Se implementaron versiones comparables, con commits que identifican al asistente.

4. Incidencia real: pytest no disponible; pip no pudo conectarse por restricciones de red. Se migró la suite a unittest y se ejecutó. No se conserva una captura de ese error; este registro es una descripción de la incidencia, no una transcripción literal de la consola.

5. Validación: registros completos de las ejecuciones por hito en evidencias/pruebas; comparación de consolas en evidencias/demo_comparada.json; revisión final en checklist_code_review.md.

Los puntos 3 a 5 son un resumen del proceso real, no prompts nuevos inventados ni una conversación reconstruida. Los diffs y commits permiten inspeccionar los cambios realizados. El anexo del informe incluye el prompt maestro y este registro. No se dispone de una exportación íntegra de todas las llamadas internas de herramientas; no se afirma que este archivo sea una transcripción completa de la sesión.

Responsabilidad y límites

El material está preparado para revisión y estudio por Gevont Utmazian y Martín Maine. No hay evidencia todavía de que los estudiantes hayan ejecutado personalmente el proyecto o comprendido todos los cambios. Las conclusiones y el guion son borradores para revisar, no constancias de aprendizaje ya demostrado. No se generó video ni voz artificial. No hubo revisión externa.

La consigna admite IA como apoyo; el grado de generación aquí declarado es amplio y debe ser revisado por los estudiantes según las condiciones de la cátedra. No se presume autorización académica adicional.

12. Referencias y recursos

[R1] Python Software Foundation. Documentación de unittest, Python 3.10. Consulta: 01/11/2026. https://docs.python.org/3.10/library/unittest.html

[R2] Python Software Foundation. Documentación de sqlite3, Python 3.10. Consulta: 01/11/2026. https://docs.python.org/3.10/library/sqlite3.html

[R3] Martin Fowler. Refactoring Catalog: Extract Function (alias Extract Method). Consulta: 01/11/2026. https://refactoring.com/catalog/extractFunction.html

[R4] Martin Fowler. Refactoring Catalog: Introduce Parameter Object. Consulta: 01/11/2026. https://refactoring.com/catalog/introduceParameterObject.html

Los términos Extract Method e Introduce Parameter Object se emplean según esas entradas del catálogo [R3, R4]. Los fragmentos del informe son del proyecto, no copias de los ejemplos del sitio.

[C1] Cátedra de Metodología de Sistemas II. TP MS2V2.pdf, seis páginas, suministrado por el usuario. Fuente principal de requisitos.

[C2] Material complementario suministrado como Markdown pegado(1).md. Incluye orientaciones de entrega y defensa.

Repositorio GitHub: proyectofinal_met2.

Video grupal: PENDIENTE DE GRABACIÓN Y ENLACE (todos los integrantes con cámara encendida, rostro y nombre completo).

Fecha de entrega: 08/11/2026. Participación de cada integrante: ver sección 2.1.

13. Auditoría y cierre de entrega

Matriz de cumplimiento

Requisito	Estado	Evidencia y archivo	Observación / acción
Sistema inicial funcional	Cumplido	estado-inicial; 01_inicial.txt	29 pruebas ejecutadas
Tres entidades y cuatro operaciones	Cumplido	sistema inicial, cli.py	Tres tablas; ocho operaciones
Persistencia	Cumplido	test_persistencia	Cierre y reapertura real
Antes/después	Cumplido	versiones/inicial; demo_comparada.json	Diez pasos idénticos
Ocho debilidades	Cumplido	docs/diagnostico.md; informe secciones 4 y 5	Ubicación, impacto, riesgo y prioridad
SOLID y Clean Code	Cumplido	docs/principios_y_decisiones.md	Aplicación justificada y límites explícitos
Cuatro refactorings	Cumplido	docs/refactorings.md; commits	Pruebas ejecutadas por cambio
Pruebas y regresión	Cumplido	evidencias/pruebas	33 pruebas finales; sin regresiones conocidas en casos cubiertos
Checklist y correcciones	Cumplido	docs/checklist_code_review.md	Autorrevisión asistida; no independiente
Registro de IA	Cumplido	docs/registro_ia.md; prompt_maestro.txt	Intervención sustancial; sin exportación completa de herramientas
Informe único editable	Cumplido localmente	informe/ PDF y Markdown	Enlaces externos siguen pendientes
Anexo de evidencias	Cumplido	evidencias/ y ZIP	Registros reales; no capturas de consola fabricadas
README e historial	Cumplido	README.md; historial.bundle	Recuperable sin conexión
Seis tags	Cumplido al empaquetar	historial.bundle y manifiesto_entrega.json	Se verifica contra Git antes del ZIP
ZIP ejecutable	Cumplido al verificar extracción	manifiesto_entrega.json; validacion_paquete.txt	Pruebas desde copia descomprimida
Capturas de pantalla	Pendiente	Sección 7	Hay registros de ejecución. Falta incorporar las capturas de pantalla.
Repositorio GitHub y enlace	Parcial	Sección de referencias	Repositorio creado. Falta subir y verificar el contenido.
Video y enlace	Pendiente	Guion preparado	Grabar con todos los integrantes, rostro y voz propios
Modalidad grupal y fecha	Cumplido	Portada y sección 2.1	Fecha de entrega y tareas documentadas.
Comprensión personal	No verificable	Ejercicios de defensa propuestos	Los estudiantes deben estudiar y demostrar

La rúbrica se cubre mediante diagnóstico (15%), deuda técnica (20%), principios (20%), refactoring (20%), pruebas (10%) y revisión (10%). Presentación/repositorio/defensa (5%) permanece parcial por los pendientes externos. No se asigna una calificación ni se promete aprobación. La entrega no está completa para la cátedra hasta incorporar todos los recursos requeridos.

Anexo A. Historial real de desarrollo

Este extracto cubre el desarrollo y la verificación. Los commits posteriores de documentación y los tags finales se consultan en historial.bundle. No se inventan fechas anteriores.

```
28ac710 | Crear prototipo académico y contrato de comportamiento
d62f552 | Verificar el prototipo con unittest disponible sin instalación
efcdcb4 | Documentar ocho debilidades y conservar la referencia inicial
32d2492 | Extraer operaciones y validación repetida de nombres
1cfd75c | Mover conexión y transacciones a repositorio SQLite inyectable
b5eb55f | Introducir NuevaSolicitud conservando la interfaz compatible
a379c39 | Encapsular consultas y descomponer reglas de dominio en tabla de estados
b3828db | Verificar consola equivalente, dominio y repositorio sustituible
85689e4 | Corregir aserciones para que no desaparezcan con optimización
```

Anexo B. Prompt maestro suministrado

Se conserva el texto completo extraído, normalizando únicamente saltos de línea. Es una instrucción del usuario, no una afirmación de que todas sus propuestas tecnológicas fueran requisitos de la cátedra. Los pendientes de entrega se declaran en la auditoría.

PROMPT MAESTRO — TRABAJO PRÁCTICO INTEGRADOR DE METODOLOGÍA DE SISTEMAS II

1. CONTEXTO Y OBJETIVO PRINCIPAL

Necesito que desarrolles de manera integral, organizada y técnicamente correcta el Trabajo Práctico Integrador de la asignatura **Metodología de Sistemas II**, correspondiente a la Tecnicatura Universitaria en Programación a Distancia de la Universidad Tecnológica Nacional (UTN).

Voy a proporcionarte dos documentos fundamentales:

1. El PDF con la consigna oficial del Trabajo Práctico Integrador.

2. Un documento Markdown con información complementaria, aclaraciones, criterios de evaluación y orientaciones de la cátedra.

Antes de comenzar, debés leer y analizar ambos documentos completos.

La consigna oficial será la fuente principal para determinar los requisitos obligatorios. El material complementario servirá para interpretar y ampliar esos requisitos.

No quiero que simplemente redactes un informe teórico. Necesito que desarrolles un trabajo académico completo, con un sistema funcional, código fuente, diagnóstico técnico, refactorización, pruebas ejecutadas, documentación y evidencias verificables.

El resultado debe ser coherente, reproducible y suficientemente claro como para que yo pueda comprenderlo, ejecutarlo, explicarlo y defender cada decisión tomada.

Datos académicos

- Universidad: Universidad Tecnológica Nacional (UTN).

- Carrera: Tecnicatura Universitaria en Programación a Distancia.

- Asignatura: Metodología de Sistemas II.

- Trabajo: Trabajo Práctico Integrador.

- Estudiante: Gevont Utmazian.

- Comisión: M25 C4-08.

- Año académico: 2026.

- Modalidad prevista: individual, sujeta a las condiciones de autorización establecidas por la cátedra.

No inventes nombres de docentes, fechas de entrega, autorizaciones, calificaciones ni datos institucionales que no estén confirmados.

Si algún dato administrativo es indispensable y no está disponible, dejalo claramente identificado para su posterior confirmación.

2. FORMA DE TRABAJO Y CRITERIOS GENERALES

Quiero que trabajes como un desarrollador de software y analista de calidad con experiencia, pero adaptando el nivel de complejidad al de un estudiante universitario.

El proyecto tiene que demostrar conocimientos reales de la materia, sin convertirse en un sistema excesivamente sofisticado.

Principios que debés respetar

1. Cumplir todos los requisitos obligatorios de la consigna.

2. Priorizar la calidad técnica sobre la cantidad de funcionalidades.

3. Desarrollar un sistema sencillo, pero funcional y verificable.

4. Mantener una relación clara entre los problemas detectados y las mejoras aplicadas.

5. Demostrar mediante pruebas que las modificaciones no introdujeron regresiones.

6. Utilizar explicaciones técnicas comprensibles.

7. Evitar complejidad arquitectónica innecesaria.

8. Documentar las decisiones verdaderamente importantes.

9. Mantener consistencia entre código, capturas, tablas, informe y repositorio.

10. No inventar pruebas, resultados, capturas, commits o evidencias.

11. Trabajar de manera progresiva y verificar cada etapa.

12. No considerar terminado el proyecto hasta completar una auditoría de requisitos.

No quiero recibir solamente recomendaciones, pseudocódigo o ejemplos aislados. Necesito los archivos reales del proyecto, siempre que el entorno de trabajo permita crearlos y ejecutarlos.

Cuando alguna operación no pueda realizarse, explicá concretamente qué falta y cómo completarla.

3. ESTILO DE REDACCIÓN: NATURAL, ACADÉMICO Y REALISTA

Este punto es especialmente importante.

Quiero que el informe tenga una redacción natural, fluida y propia de un estudiante universitario argentino que comprende los conceptos de la asignatura.

El texto debe ser académico, pero no innecesariamente formal.

Características de la redacción

- Utilizar español rioplatense adecuado para una entrega universitaria.

- Escribir con claridad, precisión y naturalidad.

- Explicar las decisiones con ejemplos concretos del sistema desarrollado.

- Evitar afirmaciones grandilocuentes.

- Utilizar párrafos de extensión variable.

- No repetir constantemente las mismas estructuras.

- Relacionar los conceptos teóricos con el código del proyecto.

- Mantener continuidad entre apartados.

- Usar terminología técnica cuando corresponda, explicándola de forma comprensible.

- Reconocer las limitaciones reales del desarrollo.

Evitar expresiones artificiales o repetitivas

No abusar de frases como:

- "En el presente trabajo práctico..."

- "Cabe destacar que..."

- "Es importante mencionar que..."

- "En un mundo cada vez más digitalizado..."

- "En conclusión, podemos afirmar..."

- "Esta solución innovadora y robusta..."

- "Se garantiza una arquitectura altamente escalable..."

Tampoco quiero que todos los párrafos comiencen con expresiones similares.

Ejemplo del estilo esperado

En lugar de escribir:

"Se procedió a implementar una refactorización mediante la cual se optimizó significativamente la mantenibilidad y escalabilidad del sistema."

Preferir algo como:

"Durante la revisión encontramos que una misma función se encargaba de validar los datos, registrar la solicitud y mostrar los resultados. Esto hacía que el código fuera difícil de modificar. Para resolverlo, separamos esas responsabilidades en funciones más pequeñas y comprobamos que el sistema siguiera funcionando como antes."

Ese es el estilo que busco: concreto, técnico y natural.

La redacción debe corresponderse con el trabajo realmente realizado. No atribuyas experiencias, decisiones manuales o pruebas al estudiante si no ocurrieron.

El uso de inteligencia artificial debe declararse honestamente, tal como exige la consigna.

4. SELECCIÓN DEL SISTEMA A DESARROLLAR

Si no te proporciono un sistema existente, utilizá el caso sugerido por la cátedra:

Sistema de Gestión de Solicitudes Académicas.

Este sistema permitirá gestionar solicitudes realizadas por estudiantes de una institución educativa.

La idea es desarrollar una aplicación pequeña que permita registrar solicitudes, asignar responsables, modificar estados, consultar información y generar reportes.

No necesito una plataforma empresarial ni una aplicación web compleja.

El objetivo es disponer de un sistema suficientemente completo para analizar su calidad interna y mejorarlo mediante refactoring.

4.1. Alcance funcional

El sistema debe implementar, como mínimo:

1. Registrar estudiantes.

2. Registrar solicitudes académicas.

3. Clasificar solicitudes por tipo.

4. Establecer prioridades.

5. Asignar responsables.

6. Modificar el estado de una solicitud.

7. Consultar solicitudes registradas.

8. Buscar y filtrar solicitudes.

9. Generar un reporte básico.

Estas operaciones pueden combinarse si el diseño del sistema lo justifica.

4.2. Entidades principales

Utilizar como mínimo las siguientes:

- Estudiante.

- Solicitud.

- Responsable o Área.

- Estado.

Podés representar tipo y prioridad mediante enumeraciones o estructuras equivalentes.

Las relaciones entre entidades deben ser coherentes y estar documentadas.

4.3. Persistencia

El sistema debe guardar o simular correctamente los datos.

Como opción inicial, utilizar:

- Python 3.11 o superior.

- SQLite para persistencia.

- Una interfaz de consola sencilla.

- pytest para las pruebas automatizadas.

- Git para control de versiones.

Esta tecnología es una propuesta, no una exigencia de la cátedra.

Si encontrás una alternativa más apropiada para el contexto académico, explicá sus ventajas antes de adoptarla.

La aplicación debe funcionar localmente, sin depender de servicios pagos o configuraciones externas complejas.

Utilizá únicamente información ficticia o anonimizada.

5. CONSTRUCCIÓN DE LA VERSIÓN INICIAL

La consigna exige trabajar sobre un sistema existente o un prototipo inicial.

Si creás el prototipo específicamente para esta actividad, indicá ese origen con transparencia.

Primero desarrollá una versión base que funcione, pero que presente debilidades técnicas reales y analizables.

No necesito que el programa falle deliberadamente.

La aplicación inicial debe ejecutar sus operaciones correctamente y permitir observar problemas en la organización del código.

Esos problemas pueden surgir de decisiones de diseño simples que resulten difíciles de mantener.

Características de la versión inicial

- Al menos tres entidades o estructuras principales.
- Al menos cuatro operaciones funcionales verificables.
- Persistencia o simulación coherente de datos.
- Funcionalidades ejecutables.
- Problemas de diseño identificables.
- Comportamiento reproducible para compararlo con la versión final.

Posibles problemas a analizar

- Funciones demasiado extensas.
- Código duplicado.
- Responsabilidades mezcladas.
- Nombres poco descriptivos.
- Dependencias innecesariamente fuertes.
- Validaciones repetidas.
- Parámetros excesivos.
- Condicionales complejos.
- Valores mágicos.
- Manejo de errores inconsistente.

No introduzcas problemas absurdos únicamente para completar la cantidad exigida.

Los defectos seleccionados deben poder explicarse y corregirse de manera razonable.

Antes de refactorizar, guardá un estado inicial verificable del proyecto.

6. DESARROLLO DE LAS OCHO ETAPAS OBLIGATORIAS

ETAPA 1 — Selección y descripción del sistema base

Elaborá una descripción completa del sistema seleccionado.

Debe incluir:

- Nombre del sistema.
- Problema que busca resolver.
- Contexto de utilización.
- Usuarios y actores.
- Alcance del proyecto.
- Funcionalidades principales.
- Entidades y relaciones.
- Tecnologías utilizadas.
- Justificación de las tecnologías.

- Estructura inicial del código.

- Forma de ejecución.

- Limitaciones funcionales.

Incluí un esquema o diagrama sencillo cuando ayude a comprender la solución.

Documentá el funcionamiento de la versión inicial mediante capturas auténticas o salidas verificables.

Explicá por qué este sistema resulta adecuado para aplicar los contenidos de la materia.

ETAPA 2 — Calidad externa y calidad interna

Analizá la diferencia entre ambos conceptos utilizando el sistema desarrollado.

Calidad externa

Evaluá aspectos observables por el usuario:

- Funcionamiento de las operaciones.

- Respuestas ante entradas válidas e inválidas.

- Claridad de los mensajes.

- Consistencia de los resultados.

- Facilidad de uso.

- Comportamiento esperado.

Calidad interna

Evaluá aspectos relacionados con el código:

- Legibilidad.

- Mantenibilidad.

- Cohesión.

- Acoplamiento.

- Responsabilidades.

- Organización modular.

- Reutilización.

- Facilidad de prueba.

- Facilidad para incorporar modificaciones futuras.

Elaborá una tabla comparativa que muestre situaciones concretas.

Por ejemplo, una función puede registrar correctamente una solicitud desde la perspectiva del usuario, pero internamente mezclar validaciones, persistencia y presentación.

Explicá por qué ese problema es relevante, aunque la aplicación aparentemente funcione bien.

No quiero una sección formada solamente por definiciones teóricas.

ETAPA 3 — Deuda técnica y code smells

Identificá **al menos ocho problemas técnicos reales** presentes en la versión inicial.

Para cada problema, documentá:

1. Identificador.

2. Nombre del code smell.

3. Archivo y ubicación.

4. Fragmento de código original.

5. Descripción del problema.

6. Evidencia concreta.

7. Impacto sobre la calidad.

8. Riesgo de mantenimiento.

9. Prioridad de corrección.

10. Mejora propuesta.

11. Relación con los refactorings posteriores.

Utilizá una tabla resumen y después desarrollá los casos que requieran mayor explicación.

Clasificá los problemas por prioridad alta, media o baja.

Justificá esa clasificación.

No todos los problemas tienen que tener la misma gravedad.

Es fundamental mantener trazabilidad entre cada problema detectado y las mejoras implementadas.

Si alguno se decide no corregir, explicá por qué.

ETAPA 4 — Aplicación de SOLID y Clean Code

Analizá los cinco principios SOLID:

- SRP: Single Responsibility Principle.

- OCP: Open/Closed Principle.

- LSP: Liskov Substitution Principle.

- ISP: Interface Segregation Principle.

- DIP: Dependency Inversion Principle.

Evaluá cuáles corresponden realmente al diseño del sistema.

No fuerces herencia, interfaces o abstracciones artificiales solamente para afirmar que se aplicaron los cinco principios.

Cuando un principio no resulte especialmente relevante, explicá por qué.

Mejoras esperadas

- Separación de responsabilidades.

- Nombres más descriptivos.

- Métodos de tamaño razonable.

- Eliminación de duplicaciones.

- Organización modular.

- Validaciones consistentes.

- Tratamiento adecuado de errores.

- Reducción del acoplamiento.

- Mejora de la cohesión.

- Claridad en las dependencias.

Para cada mejora importante, documentá:

Problema identificado → principio aplicado → modificación realizada → beneficio obtenido.

Mostrá ejemplos concretos del código anterior y del mejorado.

No utilices afirmaciones genéricas sobre SOLID sin relacionarlas con la implementación.

ETAPA 5 — Refactoring seguro

Aplicá **al menos cuatro refactorings reales y documentados**.

La consigna exige incluir:

1. Extract Method.

2. Move Method o reubicación equivalente de responsabilidades.

3. Introduce Parameter Object o una simplificación equivalente de parámetros.

4. Una técnica adicional apropiada para los problemas detectados.

La cuarta técnica puede ser, por ejemplo:

- Extract Class.

- Replace Magic Number with Symbolic Constant.

- Decompose Conditional.

- Replace Conditional with Polymorphism.

Seleccioná la técnica según las necesidades reales del código.

Para cada refactoring

Registrá:

- Nombre de la técnica.

- Problema que resuelve.

- Ubicación original.

- Fragmento de código anterior.

- Código refactorizado.

- Explicación del cambio.

- Principio de diseño relacionado.

- Beneficio esperado.

- Pruebas ejecutadas.

- Resultado de la verificación.

- Commit o referencia donde se implementó.

Es muy importante distinguir entre refactoring y agregado de funcionalidad.

No mezcles cambios de comportamiento con cambios puramente estructurales sin documentarlos por separado.

Realizá modificaciones pequeñas y verificables.

Luego de cada cambio significativo, ejecutá las pruebas correspondientes.

ETAPA 6 — Pruebas y verificación del comportamiento

Necesito evidencia de que el sistema sigue funcionando correctamente después de las modificaciones.

Prepará un conjunto de pruebas que cubra las operaciones principales y los casos límite.

Como objetivo de calidad, intentá incluir al menos doce casos de prueba relevantes.

Casos sugeridos

1. Registrar estudiante válido.

2. Registrar solicitud válida.

3. Rechazar una solicitud con datos obligatorios faltantes.

4. Asignar un responsable.

5. Cambiar el estado de una solicitud.

6. Rechazar un cambio de estado inválido.

7. Consultar solicitudes.

8. Filtrar por prioridad.

9. Filtrar por estado.

10. Generar reporte.

11. Recuperar información persistida.

12. Verificar comportamiento ante identificadores inexistentes.

Podés agregar o adaptar casos según la implementación.

Tabla de pruebas

Para cada caso, registrar:

- Identificador.

- Funcionalidad.

- Datos de entrada.

- Resultado esperado.

- Resultado obtenido antes.

- Resultado obtenido después.

- Estado de la prueba.

- Evidencia disponible.

Siempre que sea posible, automatizá las pruebas con pytest.

Ejecutá efectivamente los tests.

No inventes mensajes de consola ni resultados.

Incluí capturas o registros auténticos de ejecución.

Si una prueba falla, investigá la causa y corregí el problema antes de continuar.

Explicá cómo se verificó la ausencia de regresiones funcionales.

Si utilizás cobertura de código, indicá el comando empleado y el resultado real.

ETAPA 7 — Code review profesional

Elaborá un checklist técnico de revisión.

Debe contemplar, como mínimo:

- Legibilidad.

- Nomenclatura.

- Responsabilidades.

- Duplicación.

- Complejidad.

- Manejo de errores.

- Validaciones.

- Cohesión.

- Acoplamiento.

- Principios SOLID.

- Organización de archivos.

- Persistencia.

- Pruebas.

- Documentación.

- Seguridad básica y datos sensibles.

- Mantenibilidad.

Aplicá el checklist al repositorio real.

Registro de revisión

Para cada observación relevante, indicá:

- Identificador.

- Archivo revisado.

- Problema observado.

- Severidad.

- Recomendación.

- Acción correctiva.

- Resultado.

- Estado final.

Incluí feedback técnico breve y objetivo.

Establecé criterios claros de aceptación.

Como se trata inicialmente de una entrega individual, si la revisión es realizada por el propio autor, identificala como **autorrevisión técnica**.

No inventes compañeros, revisores externos ni aprobaciones de terceros.

ETAPA 8 — Conclusiones y aprendizaje

Elaborá conclusiones conectadas directamente con el trabajo realizado.

Desarrollá:

- Principales problemas identificados.

- Cambios más importantes.

- Beneficios observados.

- Dificultades técnicas reales.

- Decisiones que resultaron acertadas.

- Limitaciones del sistema.

- Aspectos que podrían mejorarse.

- Relación con los contenidos de la materia.

- Aprendizajes obtenidos.

Redactá una conclusión individual a nombre de Gevont Utmazian.

No atribuyas experiencias personales que no hayan sido verificadas.

La conclusión debe reflejar que el objetivo del trabajo no era desarrollar muchas funcionalidades, sino mejorar la calidad interna de un sistema manteniendo su funcionamiento.

7. CONTROL DE VERSIONES Y REPOSITORIO GITHUB

Organizá el proyecto para que pueda publicarse en un repositorio GitHub.

Utilizá nombres de archivos y directorios claros.

El historial debe reflejar de manera comprensible la evolución del desarrollo.

Tags obligatorios

Respetá exactamente los nombres exigidos por la consigna:

Tag	Estado que debe identificar
`estado-inicial`	Sistema base original
`etapa-diagnostico`	Diagnóstico técnico documentado
`etapa-refactoring`	Refactorizaciones realizadas
`etapa-pruebas`	Pruebas y verificaciones
`etapa-docs`	Documentación

Tag	Estado que debe identificar
`estadorefactorizado`	Estado final del sistema

No cambies la escritura de esos tags.

Verificá que cada uno apunte al commit correspondiente.

No etiquetes diferentes etapas de manera incorrecta solamente para cumplir con el listado.

Historial de commits

Utilizá mensajes descriptivos y naturales.

Ejemplos orientativos:

- Versión inicial del sistema de solicitudes.

- Incorporación del diagnóstico de calidad.

- Separación de responsabilidades en solicitudes.

- Refactorización del manejo de parámetros.

- Incorporación de pruebas automatizadas.

- Documentación y revisión técnica final.

No simules commits anteriores ni alteres el historial para aparentar actividades inexistentes.

Prepará el repositorio local con sus commits y tags. No publiques ni sobrescribas repositorios remotos sin mi autorización.

README.md

Debe incluir:

1. Nombre del proyecto.

2. Descripción general.

3. Objetivo académico.

4. Funcionalidades.

5. Tecnologías utilizadas.

6. Requisitos de instalación.

7. Instrucciones de ejecución.

8. Instrucciones para correr las pruebas.

9. Estructura de carpetas.

10. Descripción de la versión inicial y la final.

11. Explicación de los tags.

12. Autor.

13. Comisión.

14. Referencia al informe y demás evidencias.

Las instrucciones deben ser suficientemente claras para ejecutar el proyecto desde cero en otra computadora.

8. INFORME ACADÉMICO COMPLETO

Prepará un informe académico final en PDF.

Siempre que sea posible, generá también una versión editable en LaTeX o en un formato equivalente.

El PDF tiene que estar correctamente compuesto, con buena presentación, numeración de páginas, títulos consistentes y tablas legibles.

Estructura del informe

Portada

Incluir:

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Metodología de Sistemas II

Trabajo Práctico Integrador

Evolución profesional de un sistema existente: calidad, deuda técnica, refactoring y revisión técnica

Estudiante: Gevont Utmazian

Comisión: M25 C4-08

Año: 2026

Fecha: confirmar antes de la entrega.

Índice

Generar un índice con numeración de páginas.

1. Resumen ejecutivo

Redactar entre cinco y siete líneas.

Debe sintetizar el sistema analizado, el problema principal, las mejoras implementadas y la verificación realizada.

2. Introducción y objetivos

Explicar el contexto académico, el propósito del trabajo y los objetivos del análisis.

3. Descripción del sistema base

Incorporar dominio, alcance, actores, tecnologías, funcionalidades y estructura inicial.

4. Calidad externa e interna

Incluir conceptos, análisis concreto y comparación.

5. Deuda técnica y code smells

Presentar al menos ocho problemas con evidencia, impacto y prioridad.

6. Aplicación de SOLID y Clean Code

Relacionar los principios con cambios verificables del código.

7. Refactorings aplicados

Documentar al menos cuatro técnicas con comparaciones antes/después.

8. Pruebas y verificación

Incluir metodología, casos, resultados reales y evidencia.

9. Code review

Presentar checklist, observaciones, correcciones y criterios de aceptación.

10. Conclusiones

Incluir reflexiones del trabajo y conclusión individual.

11. Uso de inteligencia artificial

Declarar las herramientas realmente utilizadas, su finalidad y su grado de intervención.

12. Referencias y documentación

Incluir fuentes utilizadas, documentación técnica y enlaces verificables.

13. Anexos

Incorporar evidencias, capturas, registros, diagramas y archivos complementarios.

Podés adaptar la numeración siempre que los contenidos obligatorios estén presentes.

9. REQUISITOS DE PRESENTACIÓN DEL PDF

Quiero un documento cuidado, pero sin una estética exageradamente empresarial.

Usar:

- Tipografía académica legible.

- Márgenes adecuados.

- Encabezados consistentes.

- Numeración de páginas.

- Índice automático.

- Tablas correctamente alineadas.

- Fragmentos de código con formato diferenciado.

- Capturas nítidas.

- Leyendas descriptivas.

- Referencias cruzadas cuando correspondan.

- Títulos y subtítulos bien jerarquizados.

Evitá tablas que se corten incorrectamente entre páginas.

No coloques imágenes decorativas que no aporten información.

Las capturas deben mostrar resultados reales del programa, comandos ejecutados, código o pruebas.

Verificá el PDF final visualmente para detectar:

- Texto superpuesto.

- Páginas vacías innecesarias.

- Tablas desbordadas.

- Capturas ilegibles.

- Errores de numeración.

- Imágenes cortadas.

- Referencias inexistentes.

- Fragmentos de código mal formateados.

El informe debe poder leerse de manera independiente, sin necesidad de abrir todos los archivos del repositorio para entender los resultados principales.

10. REGISTRO DEL USO DE INTELIGENCIA ARTIFICIAL

La consigna permite utilizar inteligencia artificial, pero exige declarar su utilización y conservar evidencia.

Prepará un apartado específico para este propósito.

Información a registrar

- Herramienta utilizada.

- Fecha de utilización, cuando se conozca.

- Finalidad de la consulta.

- Prompt o instrucción relevante.

- Resultado obtenido.

- Forma en que se utilizó el resultado.

- Validaciones realizadas.

- Grado de intervención.

Registrá también el presente prompt maestro como parte de las instrucciones utilizadas.

No inventes intercambios, fechas ni capturas.

Prepará un historial organizado de las interacciones reales disponibles.

Si alguna interacción no puede recuperarse, indicá esa limitación.

No afirmes que la inteligencia artificial solamente corrigió ortografía si también intervino en el diseño, la programación o la documentación.

El objetivo es cumplir con la política académica de transparencia y dejar constancia del proceso de trabajo.

11. ANEXOS Y EVIDENCIAS

Prepará una carpeta específica para las evidencias técnicas.

Como mínimo, intentá incluir:

- Capturas del sistema inicial.

- Capturas del sistema final.

- Evidencias de los code smells.

- Comparaciones de código.

- Resultados de pruebas.

- Checklist de revisión.

- Registro de refactorings.

- Diagramas utilizados.

- Historial de comandos relevantes.

- Referencias a commits y tags.

- Documentación complementaria.

- Registro de interacciones con IA.

Cada evidencia debe tener un nombre descriptivo.

Por ejemplo:

`evidencia_01_sistema_inicial.png`

`evidencia_02_code_smell_funcion_extensa.png`

`evidencia_03_extract_method.png`

`evidencia_04_pruebas_refactorizacion.png`

Los nombres son orientativos.

No fabriques capturas utilizando imágenes que imiten una consola o un editor.

Toda evidencia presentada como ejecución real debe provenir del programa o de las herramientas utilizadas.

12. VIDEO DE DEFENSA Y DEMOSTRACIÓN

La consigna solicita un video de aproximadamente 10 a 15 minutos.

Prepará todo el material necesario para poder grabarlo personalmente.

No generes voces artificiales ni simules la participación del estudiante.

La defensa debe realizarse con voz propia y cámara encendida, mostrando rostro y nombre completo.

Guion solicitado

Prepará un guion detallado, natural y fácil de explicar.

Debe estar escrito para Gevont Utmazian, utilizando expresiones comprensibles.

Dividilo aproximadamente de esta manera:

Minuto 0:00 a 1:00 — Presentación

- Nombre.

- Materia.

- Comisión.

- Objetivo general del trabajo.

Minuto 1:00 a 2:30 — Sistema base

- Qué sistema se desarrolló.

- Para qué sirve.

- Qué funcionalidades tiene.

- Demostración breve.

Minuto 2:30 a 4:30 — Diagnóstico

- Diferencia entre calidad externa e interna.

- Problemas encontrados.

- Principales code smells.

Minuto 4:30 a 7:30 — Mejoras

- Aplicación de SOLID.

- Clean Code.

- Refactorings realizados.

- Comparaciones de código.

Minuto 7:30 a 9:30 — Pruebas

- Cómo se verificó el funcionamiento.

- Ejecución de tests.

- Resultados antes y después.

Minuto 9:30 a 11:30 — Code review

- Checklist.

- Problemas revisados.

- Correcciones aplicadas.

Minuto 11:30 a 13:00 — Conclusiones

- Qué mejoró.

- Qué dificultades aparecieron.

- Qué se aprendió.

- Qué podría mejorarse.

Los tiempos son orientativos y pueden adaptarse a la duración real de la demostración.

Características del guion

Quiero poder leerlo, estudiarlo y explicarlo sin necesidad de memorizar frases excesivamente técnicas.

Utilizá un tono natural.

Indicá cuándo corresponde:

- Mostrar la pantalla.

- Abrir un archivo.

- Ejecutar un comando.

- Comparar dos fragmentos.

- Mostrar el resultado de un test.

- Enseñar un commit o tag.

Prepará también posibles preguntas del docente, con respuestas claras.

Incluí preguntas sobre:

- Calidad interna y externa.

- Deuda técnica.

- Code smells.

- SOLID.

- Clean Code.

- Refactoring.

- Pruebas de regresión.

- Code review.

- Decisiones tecnológicas.

- Limitaciones.

- Autoría y comprensión del desarrollo.

No necesito solamente un texto para leer: necesito comprender el contenido para defenderlo correctamente.

13. PRESENTACIÓN COMPLEMENTARIA

El material de la cátedra incluye un modelo orientativo de presentación.

Si resulta útil, prepará una presentación breve de aproximadamente 10 a 12 diapositivas.

No la consideres un sustituto del PDF obligatorio.

Estructura sugerida

1. Portada.

2. Objetivo del trabajo.

3. Descripción del sistema.

4. Calidad interna y externa.

5. Deuda técnica.

6. Code smells identificados.

7. Aplicación de SOLID.

8. Refactorings.

9. Comparación antes/después.

10. Pruebas y resultados.

11. Code review.

12. Conclusiones.

Utilizá poco texto y ejemplos concretos.

No llenes las diapositivas de definiciones.

Cada diapositiva debe servir como apoyo para explicar el trabajo.

14. ESTRUCTURA DE ARCHIVOS DEL PROYECTO

Organizá los archivos utilizando una estructura clara.

Podés utilizar como referencia:

```
TPI_Metodologia_Sistemas_II/
│
├── README.md
├── requirements.txt
├── .gitignore
│
├── src/
│   └── codigo_fuente/
│
├── tests/
│   └── pruebas_automatizadas/
│
├── docs/
│   ├── descripcion_sistema.md
│   ├── diagnostico_calidad.md
│   ├── code_smells.md
│   ├── principios_solid.md
│   ├── refactorings.md
│   ├── checklist_code_review.md
│   ├── decisiones_tecnicas.md
│   └── registro_ia.md
│
├── evidencias/
│   ├── sistema_inicial/
│   ├── diagnostico/
│   ├── refactoring/
│   ├── pruebas/
│   └── code_review/
│
├── informe/
│   ├── Informe_TPI_MS2_Gevont_Utmazian.pdf
│   └── fuente_editable/
│
├── defensa/
│   ├── guion_defensa.md
│   ├── preguntas_docente.md
│   └── presentacion/
│
└── entrega/
    └── TPI_MS2_Gevont_Utmazian.zip
```

Esta estructura es orientativa.

Adaptala al lenguaje y las tecnologías efectivamente utilizadas.

No crees archivos vacíos solamente para aparentar una entrega más completa.

El ZIP debe contener los recursos necesarios para revisar y ejecutar el proyecto.

El repositorio Git debe mantener los commits y tags correspondientes. Si el ZIP no incluye el historial Git, documentá cómo acceder a él mediante el repositorio.

15. AUDITORÍA DE LOS CRITERIOS DE EVALUACIÓN

Antes de finalizar, realizá una revisión completa de acuerdo con la rúbrica de la asignatura.

Criterio	Ponderación
Diagnóstico de calidad interna y externa	15 %
Deuda técnica y code smells	20 %
Aplicación de SOLID y Clean Code	20 %
Refactoring seguro	20 %
Pruebas y verificación	10 %

Criterio	Ponderación
Code review profesional	10 %
Presentación, repositorio y defensa	5 %
Total	**100 %**

Revisá cada apartado con criterio exigente.

No asignes una nota ficticia ni prometas una calificación.

Evaluá si existe suficiente evidencia para demostrar el cumplimiento de cada requisito.

Matriz final de cumplimiento

Prepará una tabla con estas columnas:

1. Requisito.

2. Estado.

3. Evidencia.

4. Archivo donde se encuentra.

5. Observaciones.

6. Acción pendiente, si corresponde.

Utilizá los estados:

- Cumplido.

- Parcialmente cumplido.

- Pendiente.

- No verificable.

No marques un requisito como cumplido si la evidencia todavía no existe.

Verificá especialmente:

- Sistema inicial funcional.

- Al menos tres entidades.

- Al menos cuatro operaciones.

- Persistencia de datos.

- Comparación antes/después.

- Al menos ocho code smells.

- Aplicación justificada de SOLID y Clean Code.

- Al menos cuatro refactorings.

- Pruebas ejecutadas.

- Ausencia de regresiones conocidas.

- Checklist de code review.

- Correcciones documentadas.

- Informe PDF completo.

- Registro del uso de IA.

- Anexo de evidencias.

- README.

- Tags obligatorios.

- ZIP final.

- Material para defensa.

- Enlaces accesibles.

- Video grabado por el estudiante.

16. METODOLOGÍA DE EJECUCIÓN

No intentes resolver todo mediante una única respuesta teórica.

Trabajá por fases, produciendo y verificando los archivos correspondientes.

Fase 1 — Análisis

Leer las consignas, identificar requisitos y preparar una matriz de cumplimiento.

Fase 2 — Sistema inicial

Desarrollar el prototipo, ejecutarlo, registrar su comportamiento y guardar la versión base.

Fase 3 — Diagnóstico

Analizar el código, identificar los ocho problemas técnicos y priorizar las mejoras.

Fase 4 — Refactorización

Aplicar los cambios gradualmente, documentarlos y conservar su trazabilidad.

Fase 5 — Verificación

Ejecutar las pruebas y comparar los resultados antes y después.

Fase 6 — Revisión técnica

Completar el checklist, registrar observaciones y corregir problemas pendientes.

Fase 7 — Documentación

Redactar el informe, incorporar evidencias y preparar el registro de IA.

Fase 8 — Entrega

Generar el PDF, organizar los archivos, preparar el ZIP y comprobar su contenido.

Fase 9 — Defensa

Preparar el guion, las posibles preguntas y la presentación complementaria.

Fase 10 — Auditoría final

Verificar todos los requisitos de la consigna y señalar únicamente los pendientes reales.

Antes de continuar de una fase a otra, comprobá que lo producido sea consistente con las etapas anteriores.

Si encontrás un error, corregilo y ejecutá nuevamente las verificaciones afectadas.

17. RESTRICCIONES IMPORTANTES

No quiero que:

- Entregues código que no haya sido comprobado cuando pueda ejecutarse.

- Inventes que se ejecutaron pruebas.

- Incluyas capturas falsas.

- Inventes referencias bibliográficas.

- Utilices datos personales reales.

- Simules una revisión realizada por un compañero.

- Afirmes que el sistema fue desarrollado previamente si se creó para este trabajo.

- Agregues funcionalidades innecesarias.

- Uses arquitecturas excesivamente complejas.

- Modifiques el comportamiento externo durante un refactoring sin justificarlo.

- Om itas las evidencias del antes y después.

- Publiques código en GitHub sin autorización.

- Inventes enlaces de descarga.

- Consideres grabado el video cuando solamente preparaste el guion.

- Ocultes el uso de inteligencia artificial.

La prioridad es entregar un trabajo real, entendible, técnicamente defendible y alineado con la consigna.

18. RESULTADO FINAL ESPERADO

Al terminar, necesito disponer de:

1. Proyecto funcional

Código fuente completo, organizado y ejecutable.

2. Versiones comparables

Estado inicial y estado refactorizado identificados mediante los tags correspondientes.

3. Diagnóstico técnico

Al menos ocho problemas documentados, con evidencia y prioridad.

4. Mejoras implementadas

Aplicación justificada de SOLID, Clean Code y al menos cuatro refactorings.

5. Pruebas

Tests automatizados o verificaciones reproducibles, con resultados auténticos.

6. Code review

Checklist aplicado, observaciones y correcciones.

7. Informe académico

PDF final completo y una fuente editable cuando sea posible.

8. Evidencias

Capturas, registros, comparaciones y documentación complementaria.

9. Repositorio

Proyecto preparado para GitHub con README, commits y tags.

10. Archivo ZIP

Paquete final organizado con los recursos correspondientes.

11. Defensa

Guion oral natural, instrucciones de demostración y respuestas para posibles preguntas.

12. Transparencia académica

Registro real del uso de inteligencia artificial y de su intervención en el proyecto.

13. Auditoría final

Matriz de cumplimiento que indique qué está terminado y qué requiere una acción adicional.

19. INSTRUCCIÓN FINAL

Quiero que desarrolles este Trabajo Práctico Integrador con el mayor nivel de calidad posible, respetando estrictamente la consigna de Metodología de Sistemas II.

El trabajo debe estar identificado a nombre de **Gevont Utmazian, comisión M25 C4-08**.

Prestá especial atención a la coherencia entre las ocho etapas obligatorias, porque el diagnóstico, las mejoras, las pruebas y las conclusiones tienen que corresponder al mismo sistema y al mismo proceso de desarrollo.

Priorizá una solución pequeña y bien resuelta.

No necesito cientos de archivos ni una aplicación exageradamente compleja. Necesito demostrar que sé analizar un sistema, identificar problemas técnicos, corregirlos de manera segura y explicar por qué las mejoras son importantes.

Quiero una redacción natural, concreta y académicamente apropiada. Todo lo que figure en el informe debe corresponderse con evidencias o decisiones técnicas que realmente puedan comprobarse.

No des por terminado el proyecto si faltan requisitos obligatorios.

Cuando una actividad dependa de mí, como confirmar una autorización, publicar un repositorio o grabar el video con mi propia voz, prepará todo lo necesario y señalá exactamente qué debo hacer.

Comenzá leyendo los dos documentos adjuntos, elaborá la matriz de requisitos y después avanzá con el desarrollo real del proyecto hasta completar todos los entregables que puedas producir y verificar.